



Departamento de Sistemas Informáticos
Universidad de Castilla-La Mancha
Campus Universitario s/n
02071 – Albacete

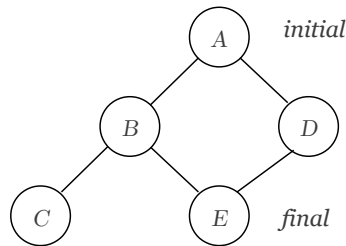
Intelligent Systems (2024/2025)

Collection of Proposed Problems

- These are some key concepts that must be clear before doing the exercises so that everybody come to the same solutions.
 - **Generated node.** Refers to either the initial node, or every node generated during the search as consequence of calling the EXPAND function.
 - **Explored node.** Refers to each node that has been extracted from the set of open nodes, regardless of the operation carried out with it next.
 - **Expanded node.** Refers to every node whose successors have been generated. All expanded nodes are explored, but not all explored nodes are expanded.
 - In TREE-SEARCH, when a node is extracted from the set of open nodes, the algorithm first tests if it is the goal. If not, then it generates the successors. In case of GRAPH-SEARCH, the algorithm first tests if it is the goal; then tests if it is already included in the set of explored states; and, afterwards, if none of the conditions holds, generates its successors.
 - Nodes are generated in alphabetic (capital before lower case), numeric, or alphanumeric order, unless another alternative is explicitly indicated. We consider that small order-nodes will be generated first. For example, if A^5 is the node to be expanded, 17 nodes have already been generated, and A can generate three successors $\{B, C, D\}$, we name them respectively as B^{18} , C^{19} and D^{20} .
 - If the strategy considers costs, we choose older nodes to break ties.
 - When using depth-limited first, nodes at level l are not expanded, only explored.
 - We consider that the depth of the root node is 0.
 - In a search tree, we represent closed nodes by surrounding them with a circle, while open nodes will not be surrounded. In the graph search version we represent the closed paths (there is a closed node representing the given state) with two parallel lines under the node (or crossing it out). A node representing the goal (if exists) must be enclosed in a square.

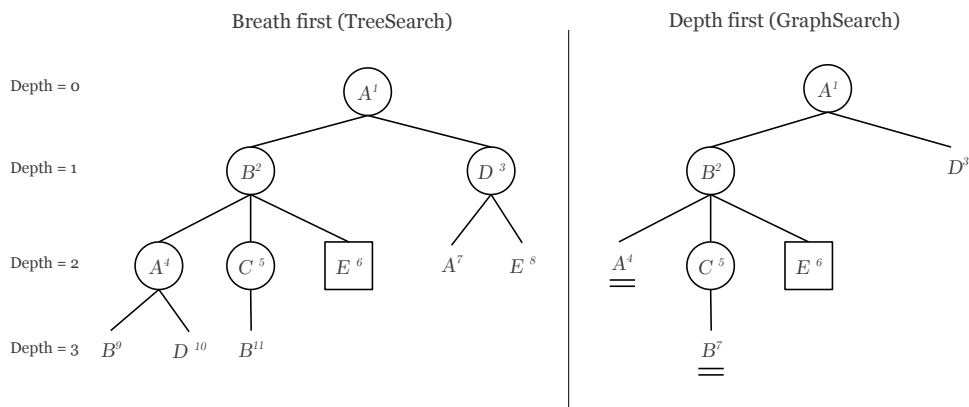
Example

Let us consider an undirected graph, where A represents the initial state and E the (only) goal state.



- Solve the problem with the TREE-SEARCH version of Breath-First search.
- Solve the problem with the GRAPH-SEARCH version of Depth-First search.
- In both cases indicate:
 - * The sequence of nodes representing the solution path found.
 - * The cost of the solution (all actions have unitary cost).
 - * The depth of the node containing the solution.
 - * The number of generated, explored, and expanded nodes.

Solution



	BF(T)	DF(G)
Solution path	ABE	ABE
Solution cost	2	2
Solution depth	2	2
Generated nodes	11	7
Explored nodes	6	6
Expanded nodes	5	3

PROBLEMS 2024/2025

1. (L2, L3) Let's assume we have the **undirected graph** (map) depicted in Figure 1 where every node represents a city and the links are labelled with the distance between the adjacent cities. Notice that an undirected edge $X - Y$ means that the route exists in both directions: $X \rightarrow Y$ and $Y \rightarrow X$. The number in the dotted box next to each node corresponds to the value for the heuristic in that city, when it is applicable. We want to use search strategies to find a path from city S to city G .

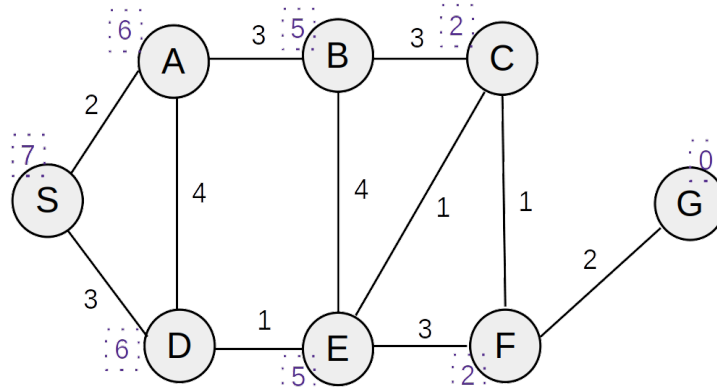


Figure 1: Map associated to problem 1.

Successors will be generated in alphabetical order, the ties which could be produced according to the search strategy (when selecting the next node from the open list) will be broken choosing always the **oldest node, following the generation order**. For all the algorithms, you are required to show the solution found, its depth, its cost, plus the number of generated, expanded and explored nodes. The search tree must be correctly annotated (that is: generation order for the nodes, states each one refers to, etc...). Those solutions not presenting enough detail will be highly penalized or, even, directly be graded as zero. You are required to:

- Apply **depth-first in tree search**.
- Apply **depth-first in graph search**.
- Apply **breadth-first in graph search**.
- Apply **iterative deepening depth-first in graph search**. Compare the results with those obtained by breadth-first.
- Apply **uniform-cost in graph search**.
- Apply **best-first in tree search**, using as heuristic the value indicated for every city/state in the map.

- (g) Apply **A* search in graph search**, using as heuristic the value indicated for every city/state in the map.
- (h) Is the given heuristic admissible? You must clearly give a "yes or no" answer, which must be appropriately justified.
2. (L2, L3) Let's assume we have this **undirected**¹ **graph**, shown in Figure 2 where every node represents a state and the links represent actions which are labelled with their cost. Also, the number in bold face next to the state corresponds to the value of the heuristic, when it is applicable. We want to use search strategies to find a path from state *S* to *G*.

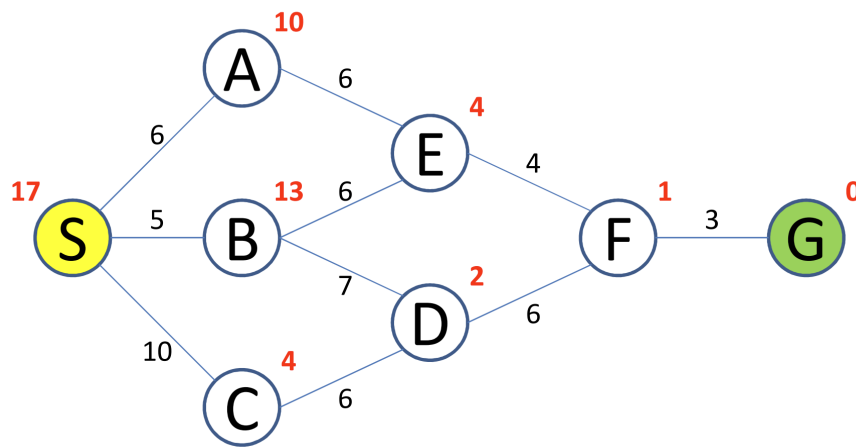


Figure 2: Graph for exercise 1

Successors will be generated in alphabetical order, the ties which could be produced according to the search strategy (when selecting the next node from the open list) will be broken choosing always the **oldest node, following the generation order**.

For all the algorithms, you are required to show the solution found, its depth, its cost, plus the number of generated, expanded and explored nodes. The search tree must be correctly annotated (that is: generation order for the nodes, states each one refers to, etc...). Those solutions not presenting enough detail will be highly penalized or, even, directly be graded as zero.

You are required to:

- (a) Apply **depth-first in graph search**.
- (b) Apply **breadth-first in graph search**.
- (c) Apply **uniform-cost in graph search**.

¹That is, an edge $X - Y$ means that the connection exists in both directions: $X \rightarrow Y$ and $Y \rightarrow X$.

- (d) Apply **iterative deepening depth-first in graph search**. Compare the results with those obtained by breadth-first.
 - (e) Apply the algorithm **greedy-first in graph search** using as a heuristic the value next to every state. Also, answer this: would it be any different performing greedy-first in tree search? Justify your answer.
 - (f) Apply the algorithm A^* **in tree search** using as a heuristic the value next to every state.
 - (g) The given heuristic is admissible, but we ask you to give a new heuristic, h_{better} , so that it is better informed, and consistent. Please, justify your answer.
3. (L4) Consider the expanded game tree shown in Fig. 3. You will apply the minimax algorithm with alpha-beta pruning to the given tree.
- You are required to:
- (a) Apply the minimax algorithm with alpha-beta pruning to the given tree. Identify clearly the output, the process, and the nodes to be pruned.
 - (b) Taking the above process into account, reorder the four actions/branches coming out of the root node (labeled B, C, D and E) so that applying minimax with alpha-beta **the number of pruned nodes is as many as possible**. Reorder only this level two! Show the reordering and the application of the minimax algorithm with alpha-beta pruning drawing the corresponding tree in your own piece of paper solving the exam. You have to provide a valid, and correctly explained justification, on how and why the branches were re-ordered
4. (L4) Consider the game tree between two players Max and Min shown in Figure 4. In the tree the numbers represent the utility value associated when reaching that node.
- (a) Apply the minimax algorithm with alpha-beta pruning to the given tree, clearly showing the result obtained by the algorithm. Clearly detail and annotate the process carried out on the tree. Indicate how many nodes would not be generated thanks to the alpha-beta pruning process.

Important: Solutions that do not sufficiently detail the process and calculations performed will be severely penalized in the grade or not scored.

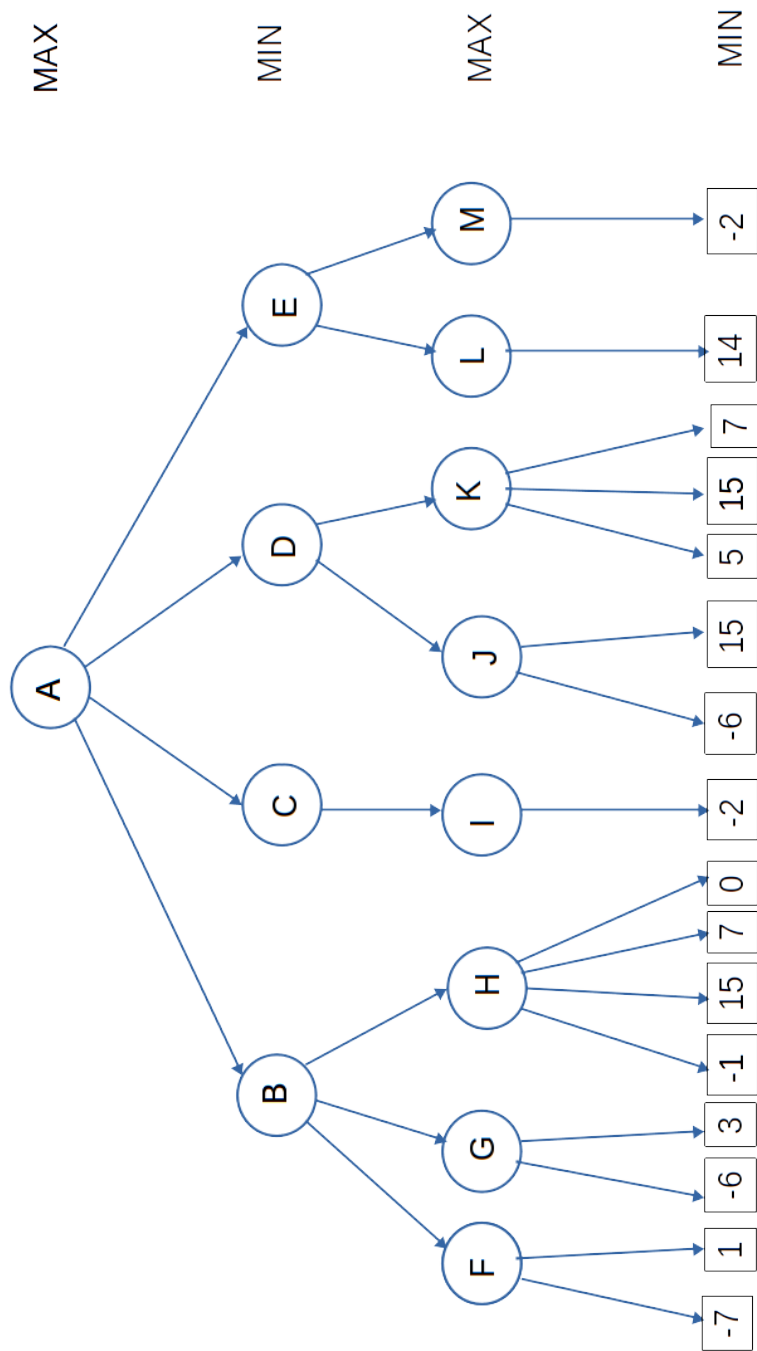


Figure 3: Game tree for minimax application with alpha-beta, for problem 3.

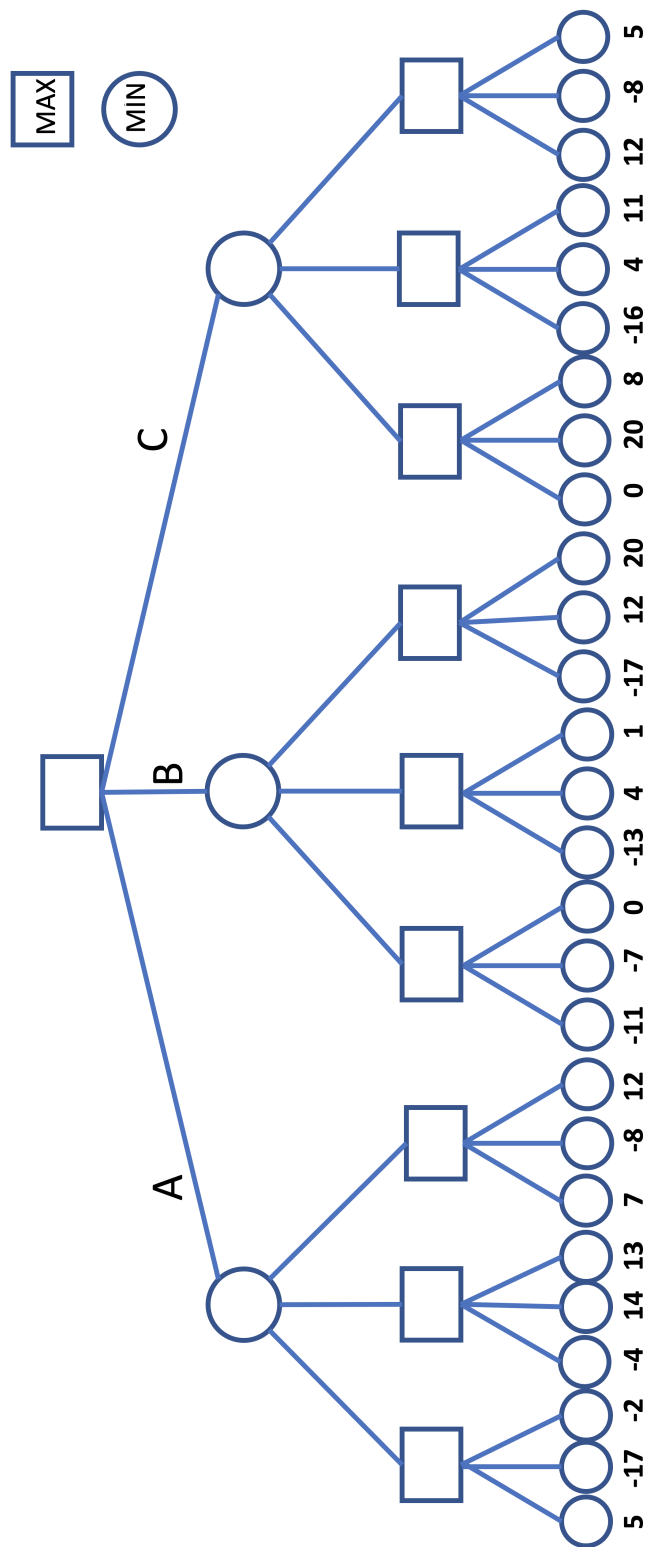


Figure 4: Game tree for the alfa-beta exercise

5. (L5) Let's consider a game in which there are three machines that are played in the following order: m_1 , m_2 and m_3 . In each one the player has two options $\{Play, NotPlay\}$, such that neither of them incurs a positive or negative cost (reward), that is, we can see that the fact of executing either action *Play* or action *NotPlay* has 0 reward.

The game proceeds as follows:

- (a) In the machine m_1 if the player chooses *NotPlay* s/he will remain in the same machine with probability 0.5 or move to the machine m_2 , also with probability 0.5. If the player decides to play then s/he receives a reward of -10 with probability 0.1 and 5 with probability 0.9. After receiving the reward the game ends.
- (b) In the machine m_2 if the player chooses *NotPlay* s/he will remain in the same machine with probability 0.5 or move to the machine m_3 , also with probability 0.5. If the player decides to play then s/he receives a reward of -10 with probability 0.2 and 10 with probability 0.8. After receiving the reward the game ends.
- (c) In the machine m_3 if the player chooses *NotPlay* s/he will remain in the same machine with probability 1. If the player decides to play then s/he receives a reward of -10 with probability 0.3 and 15 with probability 0.7. After receiving the reward the game ends.

You are required to:

- (a) Model the problem as a Markov decision model. It must clearly show the graph, the states, transitions, probabilities, etc. (Hint: use as non-final states the machines and as final states the rewards obtained when playing).
- (b) Get the optimal policy using the algorithm of value iteration. **You must complete a maximum of 3 iterations considering $\gamma=0.9$.** Can we be sure that the obtained policy is the optimal one?
- (c) Assuming the following Q-table as initial, run a complete episode considering $\alpha=0.5$ and $\gamma=0.9$, and using the random numbers you need following this order: 0.75, 0.15, 0.22, 0.25, 0.9, 0.05, ...

Q-table	Play	NotPlay
m_1	-2	0
m_2	0	0
m_3	0	0

You have to clearly describe the episode (including computations) and show the final values of the Q-table and the resulting policy.

Important: If you have to break a tie between the actions, *Play* is always preferred to *NotPlay*. If you have to sample distributions, order from highest to lowest considering the ordering that applies (numeric, lexicographic, etc.).

6. (L5) Consider the diagram in Figure 5, in which we have 3 states: $\{q_1, q_2, F\}$, with q_1 the initial state and F the final state. The local and global rewards are attached to each state. The agent can use two actions $\{A, B\}$.

The following is requested:

- Obtain the policy for the agent by using Value Iteration (5 iterations at most) with $\gamma = 1$. Show the expected utility for each state at each iteration, as well as the detailed computations. Show the obtained policy. Is the obtained policy the optimal one? Justify your answer. If an apple pie is waiting for the agent in the final state, will the agent get the pie? Justify your answer.
- Repeat the previous exercise and answer the same questions with $\gamma = 0.5$.
- Run 5 episodes for the Q-learning algorithm considering $\alpha = 0.5$ and $\gamma = 1.0$. If you need random numbers use the following in order: 0.75, 0.15, 0.22, 0.25, 0.9, 0.05, 0.75, 0.15, 0.22, 0.25, 0.9, 0.05, 0.75, 0.15, 0.22, 0.25, 0.9, 0.05, ... You should show the process in detail, the Q-table, resulting policy, etc.

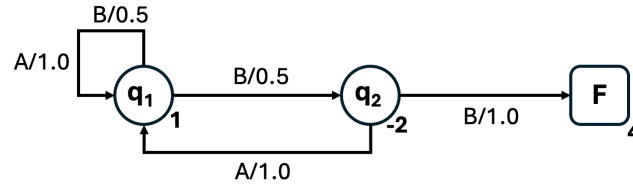


Figure 5: Markov decision process

7. (L6 - L8) Let's consider a problem where the input is a natural number $n \in \mathbb{N}$, $n \geq 2$. This indicates a square matrix with size $n \times n$ containing **all the numbers** in the series $1, 2, \dots, n^2 - 1, n^2$. We aim at designing a metaheuristic algorithm able to find the configuration of values within the matrix so that all the rows and columns sum up to a value as close as possible. For instance, if $n = 2$, we could have, among others, the following two configurations:

$$\begin{array}{cc} 1 & 2 \\ 3 & 4 \end{array} \qquad \begin{array}{cc} 1 & 4 \\ 3 & 2 \end{array}$$

where in the left matrix the additions (rows, and then columns) are $\{3, 7, 4, 6\}$ and on the left matrix they are $\{5, 5, 4, 6\}$. Then, between them two, the second one would be better as the four values are closer among them.

You are required to:

- (a) Indicate the representation that you would use to solve this problem using metaheuristics. Remember that phenotype (representation) and genotype (individual) can be different. Give two concrete examples for your representation with $n = 3$.
 - (b) Formalise the evaluation function (mathematical expression or code) and evaluate the two examples you have given in the previous section. Given your evaluation function, indicate if it is a task of maximising or minimising.
 - (c) Assuming we are going to solve the problem using genetic algorithms, what is the probability of selection for your two examples by using probability of selection proportional to fitness? Detail the calculations.
 - (d) Describe an appropriate crossover operator for your representation and apply it to your two examples. Choose the parameters of your operator so that the crossover performed is sufficiently meaningful, otherwise the score will be severely penalised.
8. (L6 - L8) We have a Sudoku game in its simplest version, 4x4, i.e., the numbers range from 1 to 4. As we know, to have a valid solution, the same number cannot be repeated either in a subarea (each of the 4 2x2 squares into which it is subdivided), or in columns, or in rows. Figure 6 shows an example:

3	1	2	4
2	4	3	1
4	2	1	3
1	3	4	2

		4	
	2		3
2			
	4		1

(a) Solved Sudoku (b) Given Sudoku

Figure 6: Sudoku example

You are asked to formulate the problem as a combinatorial optimization problem, clearly specifying:

- (a) The chosen representation. Include all the details and an example for the Sudoku presented in part (b) of Figure 6. Fill in part (b) of the figure and provide the individual/chromosome for it.

NOTE.- You have to remember that a possible solution does not have to be a valid solution; it may violate some of the constraints. The search, guided by the evaluation function, is precisely what will find the good or best possible solution.

- (b) Describe the search space and calculate its cardinality.

- (c) The evaluation function to use, first describe the idea on which it is based and then formalize it in pseudocode or with a formula.
- (d) Define a neighborhood operator to apply *Hill Climbing*. Indicate how many neighbors there would be for the example you provided in the first part. Generate a neighbor and evaluate it.
9. (L10) Consider the data set shown in Table 1, in which the provided data represents the result of a psychological experiment. It shows the registered observations of people performing one task with inflatable balloons. Each observation consists of four independent variables documenting the color of the balloon (red or blue), the size of the balloon (small or large), the activity performed with the balloon (stretch or dip), and the age of the person who handled the balloon (adult or child). The class variable is the state of the balloon after every observation: inflated (**True**) or not (**False**). Notice that the last column, Count, is not a predictive feature. It indicates how many times this observation happened. For that reason, this table represents a **total of 24 cases**. So, for instance, the first row in this compact table, would imply repeating the first observation [Red, Large, Stretch, Adult, **False**] six times, as count is 6.

Color	Size	Act	Age	Inflated	Count
Red	Large	Stretch	Adult	False	6
Red	Large	Dip	Child	False	2
Blue	Large	Stretch	Child	False	1
Blue	Large	Dip	Child	False	2
Red	Small	Dip	Child	False	2
Blue	Small	Dip	Adult	True	4
Red	Large	Stretch	Child	True	5
Blue	Large	Dip	Adult	True	2

Table 1: Dataset for problem 9. Notice it is in a compact form showing the frequencies (Count).

From these data the researchers want to build a classification tree that can be used to predict if the balloon will be inflated or not given the predictive variables {Color, Size, Act, Age}. To break the ties when selecting the variables, use the order given in the previous list. To break the ties for the prediction, **False** is preferred against **True**.

You are required to:

- (a) Build a decision tree using the procedure studied in class, **but limited to level 2** (only two decision/predictive variables at maximum). Provide all the details of the entire process and show the resulting tree completely annotated.

- (b) Using the decision tree you got previously, apply the pruning procedure studied in class. The process and computations must be clearly detailed.
- (c) On this last decision tree (maximum depth 2), apply the decision tree procedure studied in class, apply the pruning procedure studied in class. The process and calculations of this pruning, and the resulting tree, must be duly detailed and explained.
- (d) Classify the record (Experience=advanced, Motorcycle=yes, Area=rural) with the first tree learned and last tree one (obtained after applying the pruning process). Compare the results obtained and try to connect your conclusions to an important concept discussed in class for this topic.
10. (L10) An insurance company wants to construct a decision tree able to predict accident risk of a car driver based on the following attributes:

Attribute	Description	Values
Time	time since obtaining their driver's license in years	{1-2, 3-7, >7}
Gender	gender	{male, female}
Area	residential area	{urban, rural}
Risk	the risk class	{low, high}

For your analysis you have the following manually classified training examples in Table 2.

Case	Time	Gender	Area	Risk
1	1-2	m	urban	low
2	>7	f	rural	low
3	3-7	f	urban	low
4	3-7	m	urban	low
5	3-7	m	rural	high
6	1-2	f	rural	high
7	>7	m	rural	high
8	1-2	m	rural	high

Table 2: Data set to train the classifier. **Risk** is the class variable. **Case** is simply an identifier of each case considered, therefore, **should not** be considered as a predictor variable.

Our goal is to learn a classification tree from the data in Table 2, in order to predict the value of **Risk** from the data taken by the three collected predictor variables: **Time**, **Gender** and **Area**. You are required to:

- (a) Build a decision tree by applying the recursive algorithm studied in class. You must properly annotate the tree and sufficiently detail the process and calculations, otherwise the answer may be penalized or will be zero-graded.

In the leaf nodes, the notation used should be (h, l) to indicate that $h + l$ instances have arrived to that node, h of the high class and l of the low class.

In case several variables are tied applying the selection criteria, break the ties using first the criteria given in class (i.e, characteristics that generate a smaller number of rules and those that generate rules with different consequences will be favored) and if the tie remains, then use the following order: **T**, **G** and **A**. In case of having to break the tie between (high, low) to decide the label of a leaf node, it will be done in favor of high.

- (b) Apply in detail the pruning process to the tree obtained in the previous section.
- (c) Apply the decision tree to the following drivers, show their classification and the process to get them:

Case	Time	Gender	Area	Risk?
A	1-2	f	rural	
B	3-7	m	urban	
C	3-7	f	rural	